Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

Java - Concetti Base

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Livello 1 – Overview

Introduzione completa ai fondamenti del linguaggio Java.

Obiettivi del livello:

- Comprendere la struttura del linguaggio.
- Configurare l'ambiente di sviluppo.
- Usare variabili, operatori, selezioni e cicli.
- Creare i primi programmi modulari.
- Comprendere che cosa sono classi e oggetti.

enrico.viali@gmail.com

enrico.viali@gmail.com enrico.viali@gmail.com enrico.viali@gmail.com enrico.viali@gmail.com enrico.viali@gmail.com enrico.viali@gmail.com enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

1. Introduzione a Java – Parte 1

Java è un linguaggio di programmazione:

- general-purpose
- orientato agli oggetti
- portabile, grazie alla JVM
- fortemente tipizzato
- ampiamente utilizzato in applicazioni aziendali, Android, sistemi di backend.

Concetti chiave:

- Il codice sorgente Java ha estensione .java.
- Il compilatore produce bytecode (.class).
- Il bytecode viene eseguito dalla JVM (Java Virtual Machine).

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

1. Introduzione a Java – Parte 2

Componenti fondamentali dell'ecosistema:

- **JVM**: Java Virtual Machine, interpreta ed esegue il bytecode.
  - è inclusa nel JRE
- **JRE**: Java Runtime Environment, serve per eseguire i programmi.
  - di solito è incluso nel JDK
- **JDK**: Java Development Kit, contiene compilatore, strumenti, librerie.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

enrico.viali@gmail.com

1.1 Portabilità

Java cerca di essere multipiattaforma.

Essere totalmente multipiattaforma è molto difficile forse impossibile, ma in generale uno stesso programma java gira su Windows, Linux e macOS con il minimo possibile di modifiche.

Programmi semplici possono girare senza modifiche.

Possiamo quindi dire che i programmi java hanno un'alta **portabilità**.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

2. Ambiente di sviluppo – Parte 1

Per iniziare a programmare servono:

- Installazione del JDK.
- Verifica della presenza dei comandi javac e java.
- Creazione del primo file sorgente.

Per fare questo cercare e seguire un tutorial **aggiornato** in questo momento ex. tutorial aggiornato al momento in cui vengono scritte queste note:
https://www.youtube.com/watch?v=Ibi7ZFsOwF8

enrico.viali@gmail.com

Programma minimale per test

Esempio di file semplice:

creare un file di testo `primoprogramma.java` e inserire il codice seguente:

```
public class PrimoProgramma {
    public static void main(String[] args) {
        System.out.println("Benvenuto in Java");
    }
}
```


enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

2. Ambiente di sviluppo – Parte 2

Dopo aver installato e configurato il JDK è possibile compilare ed eseguire.

Compilazione e esecuzione da terminale:

```
javac PrimoProgramma.java
java PrimoProgramma
```

`javac` , il compilatore, prende in input il sorgente `PrimoProgramma.java` e se è corretto genera `PrimoProgramma.class`

`java` è la virtual machine, prende in input `PrimoProgramma.class` e lo esegue

stesso esempio con più dettagli, in Linux *(ls è il comando linux equivalente a dir del mondo DOS/Windows, *.class limita la visualizzazione ai files che terminano con .class)*

```
$ javac PrimoProgramma.java
$ ls *.class
PrimoProgramma.class
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

Punti importanti:

- Il nome della classe **deve** coincidere con il nome del file.
- L'IDE può automatizzare compilazione ed esecuzione.
- L'organizzazione di un progetto Java non è libera, in generale prevede determinate cartelle per codice, risorse e compilati.

enrico.viali@gmail.com

2. Ambiente di sviluppo – Parte 3

Per scrivere programmi Java esistono:

- editor semplici;
- IDE completi.

Un editor serve soprattutto per scrivere codice.

Un IDE aiuta anche con compilazione, esecuzione e debug.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

Editor semplici

Esempi:

- Blocco note
- Notepad++

Vantaggi:

- semplici;
- leggeri;
- utili per capire meglio compilazione ed errori.

Svantaggi:

- pochi aiuti automatici;
- meno comodi per progetti grandi.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Visual Studio Code

Visual Studio Code è molto usato oggi.

Con le estensioni Java permette:

- compilazione;
- esecuzione;
- debug;
- suggerimenti automatici.

Potenza: alta, come un IDE

Complessità: media

Alcuni riferimenti

Visual Studio Code https://code.visualstudio.com/

enrico.viali@gmail.com

BlueJ

BlueJ è un IDE pensato per studenti principianti.

Vantaggi:

- semplice;
- interfaccia chiara;
- utile per imparare classi e oggetti.

Svantaggi:

- meno adatto a progetti grandi.

Potenza: media

Complessità: bassa

Alcuni riferimenti

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

IDE professionali

Esempi:

- IntelliJ IDEA Community
- Eclipse
- NetBeans

Vantaggi:

- strumenti avanzati;
- debug potente;
- gestione grandi progetti.

Svantaggi:

- più complessi;

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

IDE - Alcuni riferimenti

IntelliJ IDEA Community https://www.jetbrains.com/idea/download/

Eclipse https://www.eclipse.org/downloads/

NetBeans https://netbeans.apache.org/

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Quale scegliere?

Nelle lezioni si utilizza un solo IDE scelto a inizio anno.

Per chi, per interesse personale, **a suo carico** vuole esaminare o usare alternative:

Per iniziare:

- Notepad++;
- Visual Studio Code + Maven (strumento di build)
- BlueJ;

Successivamente:

- IntelliJ IDEA;
- Eclipse;
- NetBeans.

enrico.viali@gmail.com

enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

3. Sintassi di base – Parte 1

Una classe Java contiene attributi (dati) e metodi (azioni).

Struttura minima:

```
public class NomeClasse {
    // attributi

    // metodi
    public static void main(String[] args) {
        // codice ...
    }
}
```

Concetti:

- Le parentesi graffe delimitano i blocchi.
- Ogni istruzione termina con il punto e virgola.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

3. Sintassi di base – Parte 2

I commenti non vengono eseguiti e servono per spiegare il codice

- Commento su una linea:

```
// testo del commento
```

- Commento su più linee:

```
/* testo su
più
righe */
```

- Commento per documentazione automatica (Javadoc):

```
/** testo che deve seguire regole prestabilite */
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

4. Tipi di dato e variabili – Parte 1

Java utilizza due categorie di tipi:

- Tipi primitivi
- Tipi reference (oggetti)

Tipi primitivi:

- interi: byte, short, int, long
- floating point: float, double
- caratteri: char
- booleani: boolean

enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

Esempi:

```
int eta = 16;
double prezzo = 12.5;
boolean attivo = true;
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

4. Tipi di dato e variabili – Parte 2

Caratteristiche:

- Una variabile deve essere dichiarata prima dell'uso.
- Il tipo determina lo spazio di memoria e le operazioni consentite.
- Le costanti si definiscono con final.

Esempio:

```
final double PI = 3.14159;
```

Le conversioni possono essere implicite (da int a double) o esplicite (double a int tramite cast).

enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

5. Operatori in Java

Categorie principali:

- Operatori aritmetici: +, -, *, /, %
- Operatori relazionali: ==, !=, <, >, <=, >=
- Operatori logici: &&, ||, !
- Operatori di assegnazione combinata: +=, -=, *=, /=
- Operatori di incremento: ++, --

Esempi:

```
int a = 5;
a += 3;       // ora a vale 8
boolean x = (a > 2) && (a < 10);
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

6. Strutture di selezione – Parte 1

La selezione permette di prendere decisioni.

Forma base:

```
if (condizione) {
    // blocco eseguito se vero
} else {
    // blocco alternativo
}
```

Esempio:

```
if (eta >= 18) {
    System.out.println("Maggiorenne");
} else {
    System.out.println("Minorenne");
}
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

6. Strutture di selezione – Parte 2

Costrutto switch:

```
switch (giorno) {
    case 1:
        System.out.println("Lunedì");
        break;
    case 2:
        System.out.println("Martedì");
        break;
    default:
        System.out.println("Altro giorno");
}
```

Regole:

- È utile quando ci sono molti casi distinti.
- Il break evita di eseguire i casi successivi.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

7. Strutture iterative – Parte 1

Java fornisce tre costrutti principali per la ripetizione.

1. Ciclo while:

```
while (condizione) {
// eseguito finché condizione è vera
}
```

2. Ciclo do-while:

```
do {
// eseguito almeno una volta
} while (condizione);
```


enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

7. Strutture iterative – Parte 2

3. Ciclo for:

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

Concetti:

- Inizializzazione: int i = 0
- Condizione: i < 10
- Aggiornamento: i++

I cicli permettono di iterare su insiemi di dati, generare sequenze, accumulare risultati.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

8. Array – Parte 1

Un array è una struttura dati di dimensione fissa che contiene elementi tutti dello stesso tipo.

Dichiarazione e creazione:

```
int[] voti = new int[5];
```

Inizializzazione:

```
int[] numeri = {1, 2, 3, 4};
```

Accesso:

```
int x = numeri[0]; // Gli indici partono da 0.
```


enrico.viali@gmail.com

8. Array – Parte 2

Scansione di un array:

```
for (int i = 0; i < numeri.length; i++) {
    System.out.println(numeri[i]);
}
```

Array multidimensionali:

```
int[][] matrice = new int[3][4];
```

Gli array sono fondamentali per archivi, statistiche e strutture dati più avanzate.

enrico.viali@gmail.com

9. Metodi – Parte 1

Un metodo rappresenta un'azione o operazione.

Struttura:

```
tipoRitorno nomeMetodo(parametri) {
    // istruzioni
    return valore;
}
```

Esempio:

```
static int somma(int a, int b) {
    return a + b;
}
```


enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

9. Metodi – Parte 2

Caratteristiche importanti:

- I metodi permettono di dividere il problema in parti più semplici.
- Migliorano la leggibilità.
- Evitano duplicazioni.

Overloading: metodi con lo stesso nome ma parametri diversi.

```
static double somma(double a, double b) {
    return a + b;
}
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

10. Introduzione a classi e oggetti – Parte 1

Java è un linguaggio orientato agli oggetti.

Concetti fondamentali:

- Classe: modello o progetto di un oggetto.
- Oggetto: istanza concreta della classe.
- Attributi: dati dell'oggetto.
- Metodi: comportamenti dell'oggetto.

Esempio di classe:

```
public class Studente {
    String nome;
    int eta;
}
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

10. Introduzione a classi e oggetti – Parte 2

Creazione di un oggetto:

```
Studente s = new Studente();
s.nome = "Marco";
s.eta = 17;
```

Costruttore:

```
public Studente(String n, int e) {
    nome = n;
    eta = e;
}
```

Uso:

```
Studente s2 = new Studente("Luca", 18);
```

Note da esaminare opzionalmente dopo aver esaurito il libro di testo. Lo studio è basato al 100% sul libro di testo.

enrico.viali@gmail.com

Conclusione Livello 1

Dopo queste lezioni è possibile:

- comprendere il funzionamento del linguaggio
- scrivere programmi strutturati e modulari
- modellare dati con classi e oggetti
- utilizzare selezioni, cicli e array

Il livello successivo introduce la programmazione orientata agli oggetti completa e gli strumenti necessari per applicazioni reali.

enrico.viali@gmail.com